Deep learning in finance

Session 5: Deep hedging

Stefano De Marco

Ecole Polytechnique

Double degree Data and Finance - March 9, 2026

In this session

- Option hedging without a parametric model: Deep Hedging

The problem (...always the same!)

- Tradable asset S
- Option with payoff $g(S_T)$, to be hedged with a self-financing portfolio.
- Grid of N points in time

$$t_j = j\,\frac{T}{N} = j\,\Delta t, \qquad j = 0, \ldots, N-1$$

The problem (...always the same!)

- Tradable asset S
- Option with payoff $g(S_T)$, to be hedged with a self-financing portfolio.
- Grid of N points in time

$$t_j = j\,\frac{T}{N} = j\,\Delta t, \qquad j = 0, \ldots, N-1$$

- Hedging portfolio: cash part + delta hedging part

$$\begin{aligned} V_t &= V_t^0 + \delta_t\, S_t \\ &= (V_t - \delta_t\, S_t) + \delta_t\, S_t \end{aligned}$$

The problem (...always the same!)

- Tradable asset S
- Option with payoff $g(S_T)$, to be hedged with a self-financing portfolio.
- Grid of N points in time

$$t_j = j\,\frac{T}{N} = j\,\Delta t, \qquad j = 0, \ldots, N-1$$

- Hedging portfolio: cash part + delta hedging part

$$\begin{aligned} V_t &= V_t^0 + \delta_t\, S_t \\ &= (V_t - \delta_t\, S_t) + \delta_t\, S_t \end{aligned}$$

- Evolution in time

$$V_{t_{j+1}} = V_{t_j} + r(V_{t_j} - \delta_{t_j} S_{t_j})\Delta t + \delta_{t_j}(S_{t_{j+1}} - S_{t_j}), \qquad j = 0, \ldots, N-1$$

The problem

- The final portfolio value is

$$
\begin{aligned}
V_T &= V_0 + \sum_{j=0}^{N-1} \left(V_{t_{j+1}} - V_{t_j} \right) \\
&= \pi_0 + \sum_{j=0}^{N-1} \left(r(V_{t_j} - \delta_{t_j} S_{t_j}) \Delta t + \delta_{t_j} (S_{t_{j+1}} - S_{t_j}) \right)
\end{aligned}
$$

where $V_0 = \pi_0$ corresponds to the initial option price (the seller's price for the option), which we also refer to as the option *premium*.

The problem

- The final portfolio value is

$$
\begin{aligned}
V_T &= V_0 + \sum_{j=0}^{N-1} \left(V_{t_{j+1}} - V_{t_j} \right) \\
&= \pi_0 + \sum_{j=0}^{N-1} \left(r(V_{t_j} - \delta_{t_j} S_{t_j}) \Delta t + \delta_{t_j} (S_{t_{j+1}} - S_{t_j}) \right)
\end{aligned}
$$

  where $V_0 = \pi_0$ corresponds to the initial option price (the seller's price for the option), which we also refer to as the option *premium*.

- **Objective**: get as close as possible to the option payoff $g(S_T)$. In other terms, we want to minimize the square of

$$\text{Hedging error} = \text{Final value of the hedging portfolio} - \text{Option payoff}$$

The problem

- The final portfolio value is

$$
\begin{aligned}
V_T &= V_0 + \sum_{j=0}^{N-1} \left(V_{t_{j+1}} - V_{t_j} \right) \\
&= \pi_0 + \sum_{j=0}^{N-1} \left(r(V_{t_j} - \delta_{t_j} S_{t_j})\Delta t + \delta_{t_j}(S_{t_{j+1}} - S_{t_j}) \right)
\end{aligned}
$$

  where $V_0 = \pi_0$ corresponds to the initial option price (the seller's price for the option), which we also refer to as the option *premium*.

- **Objective**: get as close as possible to the option payoff $g(S_T)$. In other terms, we want to minimize the square of

$$
\begin{aligned}
\text{Hedging error} &= \text{Final value of the hedging portfolio} - \text{Option payoff} \\
&= V_T - g(S_T)
\end{aligned}
$$

The problem

- The final portfolio value is

$$
\begin{aligned}
V_T &= V_0 + \sum_{j=0}^{N-1} \left(V_{t_{j+1}} - V_{t_j} \right) \\
&= \pi_0 + \sum_{j=0}^{N-1} \left(r(V_{t_j} - \delta_{t_j} S_{t_j})\Delta t + \delta_{t_j}(S_{t_{j+1}} - S_{t_j}) \right)
\end{aligned}
$$

  where $V_0 = \pi_0$ corresponds to the initial option price (the seller's price for the option), which we also refer to as the option *premium*.

- **Objective**: get as close as possible to the option payoff $g(S_T)$. In other terms, we want to minimize the square of

$$
\begin{aligned}
\text{Hedging error} &= \text{Final value of the hedging portfolio} - \text{Option payoff} \\
&= V_T - g(S_T) \\
&= \pi_0 + \sum_{j=0}^{N-1} \Delta V_j - g(S_T) \\
&= \text{option premium} + \text{contribution from hedging} - g(S_T)
\end{aligned}
$$

The problem

- **Unknowns** of the problem:
  - initial value π_0
  - delta-hedging strategy δ_t

The problem

- **Unknowns** of the problem:
  - initial value π_0
  - delta-hedging strategy δ_t
- We can use a parametric choice of functions $\pi_0 = P_\theta(0, S_0)$ and $\delta_t = \delta_\theta(t, S_t)$.

The problem

- **Unknowns** of the problem:
  - initial value π_0
  - delta-hedging strategy δ_t
- We can use a parametric choice of functions $\pi_0 = P_\theta(0, S_0)$ and $\delta_t = \delta_\theta(t, S_t)$.
- The Black-Scholes functions correspond to $\theta = \hat{\sigma}$, a one-dimensional parameter.
- We can also leverage on deep learning to tackle this problem.

Model for asset dynamics

- In any case, we need a dynamical model for the asset price S that we can simulate over the sequence of time points $(t_j)_{0 \leq i \leq N}$.

Model for asset dynamics

- In any case, we need a dynamical model for the asset price S that we can simulate over the sequence of time points $(t_j)_{0 \leq i \leq N}$.

- For today's session, we stick to the simple *log-normal dynamical model*, that is the model with independent Gaussian log-returns:

$$\begin{aligned} \log S_{t_{j+1}} &= \log S_{t_j} + \text{log-return} \\ &= \log S_{t_j} + \mu \Delta t + \sigma \sqrt{\Delta t}\, G_j \end{aligned}$$

  where

  - μ is a drift parameter
  - σ a volatility parameter
  - $(G_j)_{j \geq 1}$ a sequence of independent random variables with Gaussian distribution $\mathcal{N}(0, 1)$.

Model for asset dynamics

- In any case, we need a dynamical model for the asset price S that we can simulate over the sequence of time points $(t_j)_{0 \leq i \leq N}$.

- For today's session, we stick to the simple *log-normal dynamical model*, that is the model with independent Gaussian log-returns:

$$\begin{aligned} \log S_{t_{j+1}} &= \log S_{t_j} + \text{log-return} \\ &= \log S_{t_j} + \mu \Delta t + \sigma \sqrt{\Delta t}\, G_j \end{aligned}$$

  where

  - μ is a drift parameter
  - σ a volatility parameter
  - $(G_j)_{j \geq 1}$ a sequence of independent random variables with Gaussian distribution $\mathcal{N}(0,1)$.

- We simulate a sample of N_{train} copies of the asset price trajectory

$$\left(S^i_{t_j} \right)_{0 \leq j \leq N}^{1 \leq i \leq N_{\text{train}}}$$

  This will be our training set.

Deep hedging

- How to introduce a deep learning model into the option hedging problem?

Deep hedging

- How to introduce a deep learning model into the option hedging problem?

- **Answer** : learn the delta-hedging strategy $\delta(t, S_t)$ along the asset paths!

  And the initial option premium $\pi_0(S_0)$ as well.

Deep hedging

- We can parameterize the hedging strategy with neural networks h_{θ_j}

$$\delta_{t_j} = h_{\theta_j}(S_{t_j}), \qquad j = 0, \ldots, N-1 \tag{1}$$

  - ▶ Note we do not consider any particular time structure (e.g. ResNet) for the neural networks here.
  - ▶ We simply use a different neural network for each time point t_j.

Deep hedging

- We can parameterize the hedging strategy with neural networks h_{θ_j}

$$\delta_{t_j} = h_{\theta_j}(S_{t_j}), \qquad j = 0, \ldots, N-1 \tag{1}$$

  - ▶ Note we do not consider any particular time structure (e.g. ResNet) for the neural networks here.
  - ▶ We simply use a different neural network for each time point t_j.

- We also learn the initial option premium: $\pi_0 = \pi_\theta(S_0)$.

Deep hedging

- We can parameterize the hedging strategy with neural networks h_{θ_j}

$$\delta_{t_j} = h_{\theta_j}(S_{t_j}), \qquad j = 0, \dots, N-1 \tag{1}$$

  ▶ Note we do not consider any particular time structure (e.g. ResNet) for the neural networks here.
  ▶ We simply use a different neural network for each time point t_j.

- We also learn the initial option premium: $\pi_0 = \pi_\theta(S_0)$.

- Remark: we could consider different model specifications.

  The choice (1) is rather demanding (in terms of computational burden) because we consider a different NN at each time step t_j, and there is no connection between one NN and the others.

  Let us nevertheless start from the basic choice (1), and test its capabilities.

Deep hedging

- We can see the deep hedging problem as a supervised learning problem, where the input data x_i correspond to the trajectories in the training set $(S^i_{t_j}))^{1\leq i\leq N_{\text{train}}}_{0\leq j\leq N}$.
- The predicted data y_i correspond to the hedging error

$$\pi_\theta(S_0) + \sum_{j=0}^{N-1} h_{\theta_j}(S^i_{t_j})\big(S^i_{t_{j+1}} - S^i_{t_j}\big) - g\big(S^i_T\big)$$

For simplicity of notation, we have considered the case $r = 0$ in the evolution of the hedging portfolio above.

Deep hedging

- We can see the deep hedging problem as a supervised learning problem, where the input data x_i correspond to the trajectories in the training set $(S^i_{t_j}))^{1\leq i\leq N_{\text{train}}}_{0\leq j\leq N}$.
- The predicted data y_i correspond to the hedging error

$$\pi_\theta(S_0) + \sum_{j=0}^{N-1} h_{\theta_j}(S^i_{t_j})\big(S^i_{t_{j+1}} - S^i_{t_j}\big) - g\big(S^i_T\big)$$

  For simplicity of notation, we have considered the case $r = 0$ in the evolution of the hedging portfolio above.

- The target output data $\hat{y}_i$ is simply zero.

Deep hedging

- We can see the deep hedging problem as a supervised learning problem, where the input data x_i correspond to the trajectories in the training set $(S^i_{t_j}))^{1\leq i\leq N_{\text{train}}}_{0\leq j\leq N}$.
- The predicted data y_i correspond to the hedging error

$$\pi_\theta(S_0) + \sum_{j=0}^{N-1} h_{\theta_j}(S^i_{t_j})\big(S^i_{t_{j+1}} - S^i_{t_j}\big) - g\big(S^i_T\big)$$

  For simplicity of notation, we have considered the case $r = 0$ in the evolution of the hedging portfolio above.

- The target output data $\hat{y}_i$ is simply zero.
- The mean squared loss function is then

$$\mathcal{L}(\theta, \theta_j, S) = \sum_{i=1}^{N_{\text{train}}} \Big(\pi_\theta(S_0) + \sum_{j=0}^{N-1} h_{\theta_j}(S^i_{t_j})\big(S^i_{t_{j+1}} - S^i_{t_j}\big) - g\big(S^i_T\big)\Big)^2$$

Deep hedging

- We can see the deep hedging problem as a supervised learning problem, where the input data x_i correspond to the trajectories in the training set $(S^i_{t_j}))^{1\leq i\leq N_{\text{train}}}_{0\leq j\leq N}$.
- The predicted data y_i correspond to the hedging error

$$\pi_\theta(S_0) + \sum_{j=0}^{N-1} h_{\theta_j}(S^i_{t_j})\big(S^i_{t_{j+1}} - S^i_{t_j}\big) - g\big(S^i_T\big)$$

  For simplicity of notation, we have considered the case $r = 0$ in the evolution of the hedging portfolio above.

- The target output data $\hat{y}_i$ is simply zero.
- The mean squared loss function is then

$$\mathcal{L}(\theta, \theta_j, S) = \sum_{i=1}^{N_{\text{train}}} \Big(\pi_\theta(S_0) + \sum_{j=0}^{N-1} h_{\theta_j}(S^i_{t_j})\big(S^i_{t_{j+1}} - S^i_{t_j}\big) - g\big(S^i_T\big)\Big)^2$$

- We look for

$$\inf_{\theta,(\theta_j)_j} \mathcal{L}(\theta, \theta_j, S)$$

  over the network parameters θ and $(\theta_j)_{j=0,\ldots,N-1}$.

Option hedging: reminders and notes

- As addressed in the course Num. Proc. of financial data, the option hedging problem is often formulated in **continuous time**.
  In this case, the mathematical formalism requires continuous-time stochastic processes (such as the Brownian motion).

- If we choose the log-normal dynamical model for the underlying asset price S,

Option hedging: reminders and notes

- As addressed in the course Num. Proc. of financial data, the option hedging problem is often formulated in **continuous time**.
  In this case, the mathematical formalism requires continuous-time stochastic processes (such as the Brownian motion).

- If we choose the log-normal dynamical model for the underlying asset price S, it can be shown that, **in the continuous-time setting, the Black-Scholes delta hedging strategy provides a *perfect hedging***, in the sense that

  $$V_T = g(S_T)$$

  That is: the hedging portfolio constructed in continuous time perfectly replicates the option payoff (in the theoretical situation where we can hedge continuously in time).

Option hedging: reminders and notes

- As addressed in the course Num. Proc. of financial data, the option hedging problem is often formulated in **continuous time**.
  In this case, the mathematical formalism requires continuous-time stochastic processes (such as the Brownian motion).

- If we choose the log-normal dynamical model for the underlying asset price S, it can be shown that, **in the continuous-time setting, the Black-Scholes delta hedging strategy provides a *perfect hedging***, in the sense that

  $$V_T = g(S_T)$$

  That is: the hedging portfolio constructed in continuous time perfectly replicates the option payoff (in the theoretical situation where we can hedge continuously in time).

- In the discrete-time setting (hedging on a finite number of dates), even with the Black-Scholes price function and delta we get an hedging error, *aka* a non-zero PnL: we only get

  $$V_T \approx g(S_T)$$

  as we have seen in our simulation tests.

Option hedging: reminders and notes

- In principle, we do not expect a deep hedging model to perform better than the theoretical Black-Scholes hedging strategy, on an asset S that follows the log-normal dynamical model.
- But for a more general dynamical model for the underlying, and in a high-dimensional setting (n assets), the deep hedging approach can be competitive.

Implementation

We have two notebooks:

1. The first notebook

   Simple_learning_task_with_Keras.ipynb

   - Presents how deep models can be built using Tensorflow via a Keras interface.
   - The problem considered here has nothing to do with deep hedging: it is a simple learning problem of a one-dimensional function.

2. In the second notebook,

   Deep_hedging.ipynb

   we implement the deep hedging approach for the hedging of a call option under the log-normal dynamics.

Project 2

Part 1. Complete the list of To Do's at the the end of the notebook Deep_hedging.ipynb.

Part 2. Extension: deep hedging in the multi-asset case

- Consider two tradable assets with values S_t^1 and S_t^2. Example: two stocks in the same stock index (e.g. two components of the SP500).

- We want to delta-hedge an option with payoff $g(S_T^1, S_T^2)$ over the time grid $t_j = j\,\frac{T}{N}$, $j = 0, \ldots, N-1$.

  We consider an at-the-money (ATM) call on the product of the assets:

$$g(S_T^1, S_T^2) = \left(\frac{S_T^1 S_T^2}{S_0^1 S_0^2} - 1 \right)^+$$

Project 2. Deep hedging : the multi-asset case

- The hedging portfolio now has a delta component δ_t^i in each of the two assets:

$$V_t = V_t^0 + \delta_t^1 \, S_t^1 + \delta_t^2 \, S_t^2 = V_t^0 + \sum_{k=1}^{2} \delta_t^k \, S_t^k \, .$$

The evolution in time of this self-financing portfolio is, as usual,

$$V_{t_{j+1}} = V_{t_j} + r\Big(V_t - \sum_{k=1}^{2} \delta_t^k \, S_t^k\Big)\Delta t + \sum_{k=1}^{2} \delta_{t_j}^k \, (S_{t_{j+1}}^k - S_{t_j}^k), \quad j = 0, \ldots, N-1$$

Project 2. Deep hedging : the multi-asset case

- Dynamical model for the asset prices: a two-dimensional log-normal model

$$\begin{cases} \log S^1_{t_{j+1}} = \log S^1_{t_j} + \mu_1 \Delta t + \sigma_1 \sqrt{\Delta t}\, G^1_j \\ \log S^2_{t_{j+1}} = \log S^2_{t_j} + \mu_2 \Delta t + \sigma_2 \sqrt{\Delta t}\, G^2_j \,, \end{cases}$$

Project 2. Deep hedging : the multi-asset case

- Dynamical model for the asset prices: a two-dimensional log-normal model

$$\begin{cases} \log S^1_{t_{j+1}} = \log S^1_{t_j} + \mu_1 \Delta t + \sigma_1 \sqrt{\Delta t}\, G^1_j \\ \log S^2_{t_{j+1}} = \log S^2_{t_j} + \mu_2 \Delta t + \sigma_2 \sqrt{\Delta t}\, G^2_j \,, \end{cases}$$

  - each asset evolves with its own drift parameter μ_k and volatility parameter σ_k.

Project 2. Deep hedging : the multi-asset case

- Dynamical model for the asset prices: a two-dimensional log-normal model

$$\begin{cases} \log S^1_{t_{j+1}} = \log S^1_{t_j} + \mu_1 \Delta t + \sigma_1 \sqrt{\Delta t}\, G^1_j \\ \log S^2_{t_{j+1}} = \log S^2_{t_j} + \mu_2 \Delta t + \sigma_2 \sqrt{\Delta t}\, G^2_j \,, \end{cases}$$

  - each asset evolves with its own drift parameter μ_k and volatility parameter σ_k.
  - For every j, the two increments (G^1_j, G^2_j) form a standard Gaussian vector in $\mathbb{R}^2$ with zero mean and correlation parameter ρ, that is

$$\mathbf{G}_j := \begin{pmatrix} G^1_j \\ G^2_j \end{pmatrix} \sim \mathcal{N}_{\mathbb{R}^2}(0, C), \text{ where } C = \text{ covariance matrix } = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$$

  - The vector increments $(\mathbf{G}_0, \mathbf{G}_1, \ldots, \mathbf{G}_{N-1})$ are independent.

Project 2. Deep hedging : the multi-asset case

Approach

- We aim at learning the two-dimensional hedging strategy

$$(\delta^1_{t_j}, \delta^2_{t_j}) \quad \text{for every } t_j,$$

  using a deep hedging approach: parameterize the two deltas with neural networks.

- We also parameterize the initial option premium with a NN $\pi_\theta(S^1_0, S^2_0)$.

Project 2. Deep hedging : the multi-asset case

Approach

- We aim at learning the two-dimensional hedging strategy

$$(\delta^1_{t_j}, \delta^2_{t_j}) \quad \text{for every } t_j,$$

  using a deep hedging approach: parameterize the two deltas with neural networks.

- We also parameterize the initial option premium with a NN $\pi_\theta(S^1_0, S^2_0)$.

- As in the 1-dim application, the criterion to be minimized is the square of the final hedging error:

  hedging error = (option premium + contribution from hedging) - option payoff

  along the trajectories in the training set.

Project 2. Deep hedging : the multi-asset case

Your tasks

1. Construct a training set of N_{train} two-dimensional trajectories

$$\left(S_{t_j}^{1,i}, S_{t_j}^{2,i}\right)_{0 \leq j \leq N}^{1 \leq i \leq N_{\text{train}}}$$

   using the log-normal dynamical model.

2. Construct and train a deep hedging model.

3. Assess and display the performances of the resulting model.

Project 2. Deep hedging : the multi-asset case

Your tasks

1. Construct a training set of N_{train} two-dimensional trajectories
$$\left(S_{t_j}^{1,i}, S_{t_j}^{2,i}\right)_{0\leq j\leq N}^{1\leq i\leq N_{\text{train}}}$$
using the log-normal dynamical model.
2. Construct and train a deep hedging model.
3. Assess and display the performances of the resulting model.

In step 2, the model specification is your choice, for example:

- The deltas are given by a different NN at each time step as in the 1D case,
$$\begin{aligned}\delta_{t_j}^1 &= h_j^1\left(S_{t_j}^1, S_{t_j}^2\right)\\ \delta_{t_j}^2 &= h_j^2\left(S_{t_j}^1, S_{t_j}^2\right),\end{aligned}$$
- Other constructions are possible, such as
$$\delta_{t_j}^k = h^k\left(t_j, S_{t_j}^1, S_{t_j}^2\right),$$
where each delta δ^k is a single neural network h^k that has time t as an additional input variable.